

Foundry Local — Installation Guide

Foundry Local is Microsoft's end-to-end local AI solution for running models entirely on-device. It ships as **two separate packages** that serve different purposes:

Package	Purpose	Installs a foundry command?
CLI binary	Interactive terminal tool for browsing models, running chat, managing cache	Yes
SDK (foundry-local-sdk npm / foundry-local-sdk pip)	Programmatic library for embedding inference in your app	No

Common mistake: Running `npm install foundry-local-sdk` and expecting the **foundry** CLI to become available. The SDK is a **library**, not a CLI. The **foundry** command must be installed separately.

1. Installing the CLI

The **foundry** command-line tool is a standalone native binary — it is **not** installed by the npm or pip SDK packages.

Linux (x86_64)

```
# Download the latest CLI release
curl -sL https://github.com/microsoft/Foundry-Local/releases/download/cli-preview-0.10.0/foundry-0.10.0-linux-x64.tar.gz \
  -o /tmp/foundry-cli.tar.gz

# Extract and install
sudo mkdir -p /opt/foundry
sudo tar xzf /tmp/foundry-cli.tar.gz -C /opt/foundry

# Add to PATH (choose one)
# Option A – symlink into an existing PATH directory:
sudo ln -sf /opt/foundry/foundry /usr/local/bin/foundry

# Option B – add to your shell profile:
echo 'export PATH="/opt/foundry:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

```
# Verify
foundry --version
```

Linux (ARM64)

```
curl -sL https://github.com/microsoft/Foundry-Local/releases/download/cli-preview-0.10.0/foundry-0.10.0-linux-arm64.tar.gz \
  -o /tmp/foundry-cli.tar.gz
sudo mkdir -p /opt/foundry
sudo tar xzf /tmp/foundry-cli.tar.gz -C /opt/foundry
sudo ln -sf /opt/foundry/foundry /usr/local/bin/foundry
foundry --version
```

macOS (Apple Silicon)

```
# Option A – via Homebrew (recommended)
brew tap microsoft/foundrylocal
brew install foundrylocal

# Option B – direct download (.zip)
curl -sL https://github.com/microsoft/Foundry-Local/releases/download/cli-preview-0.10.0/foundry-0.10.0-osx-arm64.zip \
  -o /tmp/foundry-cli.zip
unzip /tmp/foundry-cli.zip -d /usr/local/bin

# Option C – direct download (.pkg installer)
curl -sL https://github.com/microsoft/Foundry-Local/releases/download/cli-preview-0.10.0/foundry-0.10.0-osx-arm64.pkg \
  -o /tmp/foundry-cli.pkg
sudo installer -pkg /tmp/foundry-cli.pkg -target /

# Verify
foundry --version
```

Windows

```
# Option A – via winget (recommended)
winget install Microsoft.FoundryLocal

# Option B – direct download (.msix)
# Download from:
#   https://github.com/microsoft/Foundry-Local/releases/tag/cli-preview-0.10.0
#   foundry-0.10.0-win-x64.msix          (standard)
```

```
# foundry-0.10.0-win-x64-winml.msix (with WinML for NPU/GPU
acceleration)

# Verify
foundry --version
```

Windows + NPU/GPU acceleration: Use the `-winml` variant of the installer (`foundry-0.10.0-win-x64-winml.msix`) or install `foundry-local-sdk-winml` via npm/pip for automatic execution provider management.

2. Installing the SDK

The SDK embeds the Foundry Local runtime directly in your application process. It does **not** require the CLI to be installed.

JavaScript / TypeScript (npm)

```
# Standard (macOS / Linux)
npm install foundry-local-sdk

# Windows with WinML hardware acceleration
npm install foundry-local-sdk-winml
```

What the install script does

The npm package has a post-install script (`script/install-standard.cjs`) that:

1. Detects your platform (`win32-x64`, `linux-x64`, `linux-arm64`, `darwin-arm64`)
2. Downloads native binaries from NuGet packages:
 - `Microsoft.AI.Foundry.Local.Core` — the core runtime
 - `Microsoft.ML.OnnxRuntime.Gpu.Linux` (Linux x64) or `Microsoft.ML.OnnxRuntime.Foundry` (other platforms) — ONNX Runtime
 - `Microsoft.ML.OnnxRuntimeGenAI.Foundry` — GenAI bindings
3. Extracts them into `foundry-local-core/<platform>/` inside the package directory

Binaries end up at (for Linux x64):

```
node_modules/foundry-local-sdk/foundry-local-core/linux-x64/
├─ libonnxruntime.so
├─ libonnxruntime-genai.so
├─ libonnxruntime-genai-cuda.so
├─ libonnxruntime_providers_cuda.so
├─ libonnxruntime_providers_shared.so
```

```
└─ libonnxruntime_providers_tensorrt.so
└─ Microsoft.AI.Foundry.Local.Core.so
```

Quick Start (JS)

```
import { FoundryLocalManager } from 'foundry-local-sdk';

const manager = FoundryLocalManager.create({
  appName: 'my-app',
  logLevel: 'info'
});

const model = await manager.catalog.getModel('qwen2.5-0.5b');
await model.download(); // downloads model on first use
await model.load();      // loads into memory

const chatClient = model.createChatClient();
const response = await chatClient.completeChat([
  { role: 'user', content: 'What is the golden ratio?' }
]);
console.log(response.choices[0]?.message?.content);

await model.unload();
```

Configuration options

Option	Description	Default
appName	Identifier for your application	(required)
logLevel	Logging verbosity: debug, info, warn, error	info
cacheDir	Directory for downloaded models	Platform-specific
port	Port for embedded web service	Auto-assigned

Python (pip)

```
# Standard (macOS / Linux)
pip install foundry-local-sdk

# Windows with WinML
pip install foundry-local-sdk-winml
```

```
from foundry_local_sdk import Configuration, FoundryLocalManager

config = Configuration(app_name="my-app")
FoundryLocalManager.initialize(config)
manager = FoundryLocalManager.instance

model = manager.catalog.get_model("qwen2.5-0.5b")
model.download()
model.load()

client = model.get_chat_client()
response = client.complete_chat([
    {"role": "user", "content": "What is the golden ratio?"}
])
print(f"Response: {response.choices[0].message.content}")

model.unload()
```

C# (.NET)

```
dotnet add package Microsoft.AI.Foundry.Local
```

Rust

See the [samples/rust/](#) directory in the GitHub repo for Rust bindings and examples.

3. CLI Usage

Once installed, the CLI provides interactive model management:

```
# Check service status
foundry service status

# Browse available models
foundry model list
foundry model list --filter device=GPU
foundry model list --filter task=chat-completion

# Get details about a specific model
foundry model info phi-4-mini

# Run a model interactively
foundry model run phi-4-mini

# Pre-download a model
```

```
foundry model download phi-4-mini

# Manage local cache
foundry cache list
foundry cache location
foundry cache remove phi-4-mini
foundry cache cd /path/to/new/cache

# Restart the service if it's not responding
foundry service restart
```

Tip: Use model aliases (like `phi-4-mini`) to let Foundry Local automatically select the best variant for your hardware. Use the full model ID to target a specific variant.

4. SDK — Embedded Web Service

The SDK can start a local OpenAI-compatible HTTP server for use with any OpenAI client library:

```
import { FoundryLocalManager } from 'foundry-local-sdk';

const manager = FoundryLocalManager.create({ appName: 'my-app' });
manager.startWebService();
console.log('Service running at:', manager.urls);

// Use with any OpenAI-compatible client pointing at the local endpoint
// ...

manager.stopWebService();
```

5. SDK — Audio Transcription

```
import { FoundryLocalManager } from 'foundry-local-sdk';

const manager = FoundryLocalManager.create({ appName: 'my-app' });
const whisperModel = await manager.catalog.getModel('whisper-tiny');
await whisperModel.download();
await whisperModel.load();

const audioClient = whisperModel.createAudioClient();
audioClient.settings.language = 'en';

// Synchronous
const result = await audioClient.transcribe('/path/to/audio.wav');
console.log('Transcription:', result.text);
```

```
// Streaming
for await (const chunk of
audioClient.transcribeStreaming('/path/to/audio.wav')) {
  process.stdout.write(chunk.text);
}

await whisperModel.unload();
```

6. SDK — Embeddings

```
const embeddingClient = model.createEmbeddingClient();

// Single input
const response = await embeddingClient.generateEmbedding(
  'The quick brown fox jumps over the lazy dog'
);
console.log(`Dimensions: ${response.data[0].embedding.length}`);

// Batch input
const batchResponse = await embeddingClient.generateEmbeddings([
  'The quick brown fox',
  'The capital of France is Paris'
]);
```

7. Troubleshooting

foundry: command not found

This is the most common issue. It means the **CLI binary** is not installed or not on your **PATH**. The npm SDK package does **not** install the CLI.

Fix: Install the CLI separately using the instructions in [Section 1](#).

Request to local service failed

The Foundry Local service may not be running or may be in a stale state.

```
foundry service restart
foundry service status
```

npm install fails on Linux/macOS with WinML errors

The `foundry-local-sdk-winml` package is **Windows-only**. Its install script downloads WinML artifacts that are unavailable on other platforms.

Fix: Use the standard package instead:

```
npm install foundry-local-sdk # NOT foundry-local-sdk-winml
```

npm install hangs or fails downloading NuGet packages

The SDK install script downloads large native binaries (~350 MB on Linux x64) from NuGet. On slow connections this can appear to hang.

Fixes:

- Ensure you have network access to api.nuget.org and pkgs.dev.azure.com
- If behind a corporate proxy, set `HTTPS_PROXY` and `HTTP_PROXY` environment variables
- The install script tries two NuGet feeds in order:
 1. <https://api.nuget.org/v3/index.json> (stable releases)
 2. https://pkgs.dev.azure.com/aiinfra/PublicPackages/_packaging/ORT-Nightly/nuget/v3/index.json (pre-release builds)

Native library loading errors on Linux

If you see errors like `libonnxruntime.so: cannot open shared object file:`

1. Verify the binary was downloaded:

```
ls node_modules/foundry-local-sdk/foundry-local-core/linux-x64/
```

2. Check `LD_LIBRARY_PATH` includes that directory, or set it explicitly:

```
export LD_LIBRARY_PATH="$PWD/node_modules/foundry-local-sdk/foundry-local-core/linux-x64:$LD_LIBRARY_PATH"
```

3. For CUDA GPU support, ensure your NVIDIA drivers and CUDA toolkit are installed and `libcudart.so` is on `LD_LIBRARY_PATH`.

GPU acceleration not working

- **Windows:** Install the `-winml` variant (`foundry-local-sdk-winml` or `foundry-0.10.0-win-x64-winml.msix`)
- **Linux x64:** The standard package includes CUDA/TensorRT provider libraries (`libonnxruntime_providers_cuda.so`, `libonnxruntime_providers_tensorrt.so`). Ensure NVIDIA drivers and CUDA are installed.
- **Linux ARM64 / macOS ARM64:** The standard package uses the `Microsoft.ML.OnnxRuntime.Foundry` NuGet package which includes CPU-only inference. GPU acceleration is not currently supported on these platforms.

8. Available CLI Releases

As of version `cli-preview-0.10.0`, the following platform binaries are available on the [GitHub releases](#) page:

File	Platform	Size
<code>foundry-0.10.0-linux-x64.tar.gz</code>	Linux x86_64	~346 MB
<code>foundry-0.10.0-linux-arm64.tar.gz</code>	Linux ARM64	~111 MB
<code>foundry-0.10.0-osx-arm64.zip</code>	macOS Apple Silicon	~55 MB
<code>foundry-0.10.0-osx-arm64.pkg</code>	macOS Apple Silicon (installer)	~55 MB
<code>foundry-0.10.0-win-x64.msix</code>	Windows x64	~27 MB
<code>foundry-0.10.0-win-x64-winml.msix</code>	Windows x64 + WinML	~28 MB
<code>foundry-0.10.0-win-arm64.msix</code>	Windows ARM64	~27 MB
<code>foundry-0.10.0-win-arm64-winml.msix</code>	Windows ARM64 + WinML	~28 MB

Note: The CLI is in **public preview**. Features and commands may change before General Availability (GA).

9. SDK vs CLI — When to Use Which

Use case	Use
Browse and discover models interactively	CLI (<code>foundry model list</code>)
Quick interactive chat with a model	CLI (<code>foundry model run phi-4-mini</code>)
Manage local model cache from terminal	CLI (<code>foundry cache list/remove</code>)
Embed local inference in a Node.js app	SDK (<code>npm install foundry-local-sdk</code>)
Embed local inference in a Python app	SDK (<code>pip install foundry-local-sdk</code>)
Embed local inference in a .NET app	SDK (<code>dotnet add package Microsoft.AI.Foundry.Local</code>)
Serve models via OpenAI-compatible HTTP API	SDK (<code>manager.startWebService()</code>)
Run models offline / in production	SDK (embedded runtime, no CLI dependency)

10. Links

- **GitHub:** <https://github.com/microsoft/Foundry-Local>
- **Documentation:** <https://learn.microsoft.com/en-us/azure/foundry-local/>
- **CLI reference:** <https://learn.microsoft.com/en-us/azure/foundry-local/reference/reference-cli>
- **SDK reference (JS):** <https://learn.microsoft.com/en-us/azure/foundry-local/reference/reference-sdk-current>
- **SDK reference (legacy):** <https://learn.microsoft.com/en-us/azure/foundry-local/reference/reference-sdk-legacy>
- **CLI releases:** <https://github.com/microsoft/Foundry-Local/releases>
- **Discord:** <https://aka.ms/foundry-local-discord>
- **Samples:** <https://github.com/microsoft/Foundry-Local/tree/main/samples>